```
#!/usr/bin/env python3
"""Phase 4: assemble the draft lexicon, run example/numeral tests, write CSV + re
import json, csv, sys
sys.path.insert(0, "/home/claude")
import arcane_lexicon_tools as T
import arcane_phase3_checker as P3

R = json.load(open("/home/claude/phase4_report.json"))
canon, roots = R["canon"], R["roots"]
disp = lambda w: w.replace("ʃ", "sh").replace("ŋ", "ng")

# ---- extra quantity-slot literals (dimensionless unit, bit): best CV outside ev
used = {e["form"] for e in canon} | {"wi", "we", "mə", "me", "ne", "nə", "re", "r
num = [e["form"] for e in canon if e["group"] == "NUM"]
cands = []
for o in T.CONSONANTS - {"ŋ", "h", "j", "w", "z"}:
    for v in "aeiou":
        w = o + v
        if w in used or T.syllabify(w)[1]: continue
        cands.append((round(min(T.dist(w, n) for n in num), 2), w))
cands.sort(reverse=True)
DIMLESS, BIT = "ʃo", "vo"
assert T.dist(DIMLESS, BIT) >= 2.0 and all(T.dist(BIT, n) >= 2.0 for n in num) an
print("literal candidates:", cands[:8], "->", DIMLESS, BIT)

rows = []
orig = {"drum": "drim", "ben": "ren", "fraja": "frata", "frama": "fraka", "graʃ":
BARE = {"mi": "mə", "ni": "nə", "ri": "rə"}
for e in canon:
    if e["form"] in BARE:
        rows.append(dict(form=f"{e['form'].upper()} (formal)", gloss=e["gloss"] +
        continue
    note = f"was {disp(orig[e['form']])}" if e["form"] in orig else ""
    rows.append(dict(form=disp(e["form"]), gloss=e["gloss"], cls=e["cls"], status
for k, w in roots.items():
    g = next(c[1] for c in T.NEW_CONCEPTS if c[0] == k); cl = next(c[2] for c in
    rows.append(dict(form=disp(w), gloss=g, cls=cl, status="PROPOSED", source="Ph
rows.append(dict(form=disp(DIMLESS), gloss="dimensionless unit (ends every numera
rows.append(dict(form=disp(BIT), gloss="bit (information unit)", cls="UNIT", stat
fused = {"mi":"SELF-point","me":"SELF-surface","mə":"SELF-volume","ni":"FIELD-poi
         "ri":"TARGET-point","re":"TARGET-surface","rə":"TARGET-volume","ra":"TAR
for f, g in fused.items(): rows.append(dict(form=f, gloss=g, cls="FUSED-REF", sta
for f, g in {"s-":"IF (clitic form of bek)","sh-":"WHILE (clitic form of zet)","f
```

```
            "wi":"LIVE (performed form of kwash)","we":"LOCKED (performed form o
    rows.append(dict(form=f, gloss=g, cls="CLITIC", status="CANON (v2.0)", reinter
with open("/mnt/user-data/outputs/Arcane_Lexicon_v4_draft.csv", "w", newline="",
    w = csv.DictWriter(f, fieldnames=["form","gloss","cls","status","source","not
print("lexicon rows:", len(rows))

# ---- legality of every spoken form in the draft lexicon
spoken = [r["form"] for r in rows if r["cls"] not in ("CLITIC",) and "(formal)" n
bad = []
for f in spoken:
    f2 = f.replace("sh", "ʃ").replace("ng", "ŋ")
    if T.syllabify(f2)[1]: bad.append(f)
ALLOWED_HOMOPHONES = {"re", "ra"}   # digit 4 / TARGET-surface, radian / TARGET-p
dups = {f for f in spoken if spoken.count(f) > 1} - ALLOWED_HOMOPHONES
print("illegal forms:", bad, "| duplicate forms:", dups)

# ---- example and numeral tests
RES = []
def rec(i, title, ok, detail=""): RES.append(dict(id=i, title=title, status="PASS

rec("P4-T01", "Every draft-lexicon form is phonotactically legal (19 consonants,
rec("P4-T02", "No two lexemes share a spoken form, except the declared slot-resol
rec("P4-T03", "Original inventory (18 consonants, 12 clusters) rejects za, zet, p
clsmap = {e["form"]: e["cls"] for e in canon}
rec("P4-T04", "No same-class pair of heads is below distance 2.0", not [p for p i
rec("P4-T05", "z and v are not frame consonants; Feet cannot contain them", ("z"
rec("P4-T06", "zet and za are lexical-only (not Foot-shaped)", (not T.is_foot("ze
rec("P4-T07", "Axis consonants alone are not syllables; CV axis forms are", T.syl

# hand-flame, performed (v2.0 example with ZA kept, numerals corrected to 0.20 m)
hf = "mel po da za shi to za ma kan ken he ha sa shi za se kas".split()
DIG = {"za","ki","to","mu","re","sa","fi","du","go","pe"}; UNI = {"ma","se","ke",
def validate(tokens):
    notes, i, ok = [], 0, True
    while i < len(tokens):
        t = tokens[i]
        if t in DIG:
            j = i
            while j < len(tokens) and (tokens[j] in DIG or tokens[j] == "shi"): j
            if j >= len(tokens) or tokens[j] not in UNI: ok = False; notes.append
            i = j + 1; continue
        if t in {"po","fo","lo","pa","ba","ta","da","ka","ga","he","ha","hu","hi"
        if T.is_foot(t) or t in fused: i += 1; continue
        ok = False; notes.append(f"unrecognised token {t}"); i += 1
    return ok, notes
ok, notes = validate(hf)
```

```
rec("P4-T08", "Hand-flame (performed) tokenises into legal Feet, roles, axes, cou
digits_ok = hf[hf.index("po")+2:hf.index("ma")+1] == ["za","shi","to","za","ma"]
rec("P4-T09", "ZA SHI TO ZA MA encodes 0.20 m (two decimal places), so the 0.200
rec("P4-T10", "kan, ken, kas, kal, mel are Foot-shaped (frame + parameter vowel +

# numeral boundary under Option B (mandatory unit, with DIMLESS)
P3.UNITS.add("DL")
pc = lambda t: len(P3.parses(t, rule_unit_required=True))
from functools import lru_cache
def parses_v4(tokens):
    toks = tuple(tokens); DIGS, UN = P3.DIGITS, P3.UNITS | {"MA"}
    @lru_cache(None)
    def N(i):
        out, j = [], i
        while j < len(toks) and toks[j] in DIGS:
            j += 1; out.append(j)
        return out
    @lru_cache(None)
    def E(i):
        out = []
        if i >= len(toks): return out
        t = toks[i]
        if t == "FRAPA":
            for j, l in E(i+1):
                for k, r in E(j): out.append((k, ("FRAPA", l, r)))
        elif t == "KWAM":
            for j, idv in ID(i+1):
                for k, b in E(j): out.append((k, ("KWAM", idv, b)))
        elif t in DIGS:
            for j in N(i):
                if j < len(toks) and toks[j] in UN: out.append((j+1, ("QTY", toks
        return out
    @lru_cache(None)
    def ID(i):                      # identifier = numeral closed by the dimensio
        return [(j+1, toks[i:j]) for j in N(i) if j < len(toks) and toks[j] == "D
    return [t for end, t in E(0) if end == len(toks)]
rec("P4-T11", "Option B: FRAPA KI DL TO MU DL has exactly one parse (1, 23)", pc(
rec("P4-T12", "Option B: FRAPA KI TO DL MU DL has exactly one parse (12, 3)", pc(
rec("P4-T13", "Option B: an unterminated numeral has no parse (FRAPA KI TO MU)",
rec("P4-T14a", "Identifier as a plain numeral: KWAM KI DL TO MU MA is NOT parsed
rec("P4-T14b", "Identifier = dimensionless-unit numeral: KWAM KI DL TO MU MA has
rec("P4-T14c", "Identifier rule: KWAM KI TO DL MU MA has exactly one parse (id 12
rec("P4-T15", "Left-handed caster frame: DRIM by the right-hand rule gives X x Y
    "component cross in caster axes gives +Z; right-hand rule negates it")

# DRIM convention check (physical): right=East, up=Up, forward=North
```

```
import numpy as np
E, U, N = np.array([1,0,0]), np.array([0,0,1]), np.array([0,1,0])   # ENU basis,
phys = np.cross(E, U)       # physical right-hand rule for (right x up)
rec("P4-T16", "Physical right x up points backward (= -forward) in a right-handed

json.dump(dict(results=RES, dimless=DIMLESS, bit=BIT), open("/home/claude/phase4_
for r in RES: print(r["id"], r["status"], r["title"], r["detail"])
print(sum(r["status"]=="PASS" for r in RES), "/", len(RES))
```